

1. Project Overview

The project develops and evaluates multiple classification models using the dataset **novagen_dataset.csv**. The target variable is **Target**.

Five model configurations are implemented and evaluated using accuracy, recall, and classification reports: Logistic Regression, K-Nearest Neighbors, Random Forest, Gradient Boosting, and a Soft Voting Classifier.

DATASET

novagen_dataset.csv

TARGET VARIABLE

Target

PROBLEM TYPE

Supervised Classification

TEST SIZE

30%

TRAINING SIZE

70%

RANDOM STATE

42

2. Dataset and Target Preparation

Dataset Loading

```
df = pd.read_csv("novagen_dataset.csv")
df.head()
```

Feature and Target Separation

```
X = df.drop("Target", axis=1)
y = df["Target"]
```

All columns except **Target** are used as input features. The **Target** column is used as the classification output.

Dataset Information	Value
Dataset File	novagen_dataset.csv
Target Column	Target
Number of Rows	<i>Not Available</i>
Number of Features	<i>Not Available</i>
Target Class Values	<i>Not Available</i>

3. Feature Preparation

Blood Group Feature Conversion

Three blood-group columns are explicitly converted to integer datatype before model training.

```
df["Blood_Group_AB"] = df["Blood_Group_AB"].astype(int)
df["Blood_Group_B"] = df["Blood_Group_B"].astype(int)
df["Blood_Group_O"] = df["Blood_Group_O"].astype(int)
```

Feature	Operation
Blood_Group_AB	Converted to integer using <code>astype(int)</code>
Blood_Group_B	Converted to integer using <code>astype(int)</code>
Blood_Group_O	Converted to integer using <code>astype(int)</code>

No other feature-engineering operations are explicitly shown in the provided code.

4. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.3,  
    random_state=42,  
    stratify=y  
)
```

Parameter	Value
Training Data	70%
Testing Data	30%
random_state	42
stratify	y

Stratification is used so that the class distribution of the target variable is preserved across the training and testing subsets as determined by the split operation.

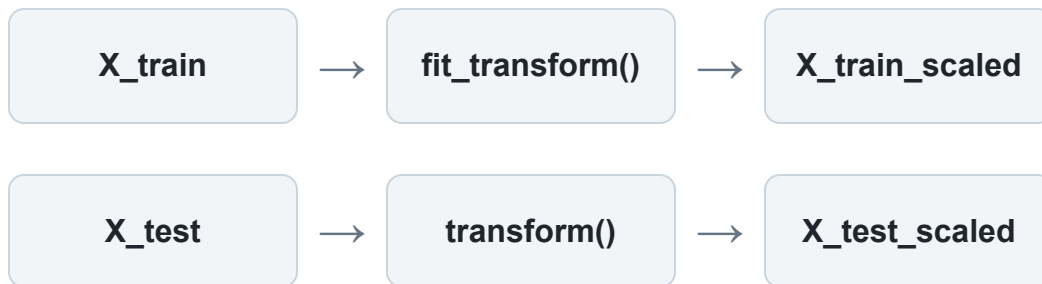
5. Feature Scaling

StandardScaler is applied before training Logistic Regression, KNN, and the Voting Classifier.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Scaling Process



The scaler is fitted only on the training data and then used to transform both the training and testing data.

6. Logistic Regression

Model Configuration

```
LogisticRegression_model = LogisticRegression(  
    penalty="l2",  
    solver="liblinear",  
    max_iter=1000  
)  
  
LogisticRegression_model.fit(X_train_scaled, y_train)
```

Parameter	Value
penalty	L2
solver	liblinear
max_iter	1000

Prediction

```
y_pred_lr = LogisticRegression_model.predict(X_test_scaled)
```

Evaluation

Accuracy, recall, and the classification report are calculated using the actual test labels and Logistic Regression predictions.

```
accuracy_score(y_test, y_pred_lr)  
  
recall_score(y_test, y_pred_lr)
```

```
classification_report(y_test, y_pred_lr)
```

7. K-Nearest Neighbors

Model Configuration

```
knn_classifier_model = KNeighborsClassifier(  
    n_neighbors=5,  
    metric="euclidean"  
)  
  
knn_classifier_model.fit(X_train_scaled, y_train)
```

Parameter	Value
n_neighbors	5
metric	Euclidean

Prediction

```
y_pred_knn = knn_classifier_model.predict(X_test_scaled)
```

Evaluation

```
accuracy_score(y_test, y_pred_knn)  
  
recall_score(y_test, y_pred_knn)  
  
classification_report(y_test, y_pred_knn)
```

8. Random Forest

Model Configuration

```
random_forest_model = RandomForestClassifier(  
    n_estimators=200,  
    oob_score=True,  
    max_depth=None  
)  
  
random_forest_model.fit(X_train, y_train)
```

Parameter	Value
n_estimators	200
oob_score	True
max_depth	None
random_state	<i>Not explicitly specified</i>

Training Data Used

Random Forest is trained directly using **X_train** and predicts using **X_test**. The scaled datasets are not used for this standalone Random Forest model.

Prediction

```
y_pred_rf = random_forest_model.predict(X_test)
```

Evaluation

```
accuracy_score(y_test, y_pred_rf)
```

```
recall_score(y_test, y_pred_rf)
```

```
classification_report(y_test, y_pred_rf)
```

9. Gradient Boosting

Model Configuration

```
Gradient_boosting_model = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=3,  
    random_state=42  
)  
  
Gradient_boosting_model.fit(X_train, y_train)
```

Parameter	Value
n_estimators	100
learning_rate	0.1
max_depth	3
random_state	42

Prediction

```
y_pred_gb = Gradient_boosting_model.predict(X_test)
```

Evaluation

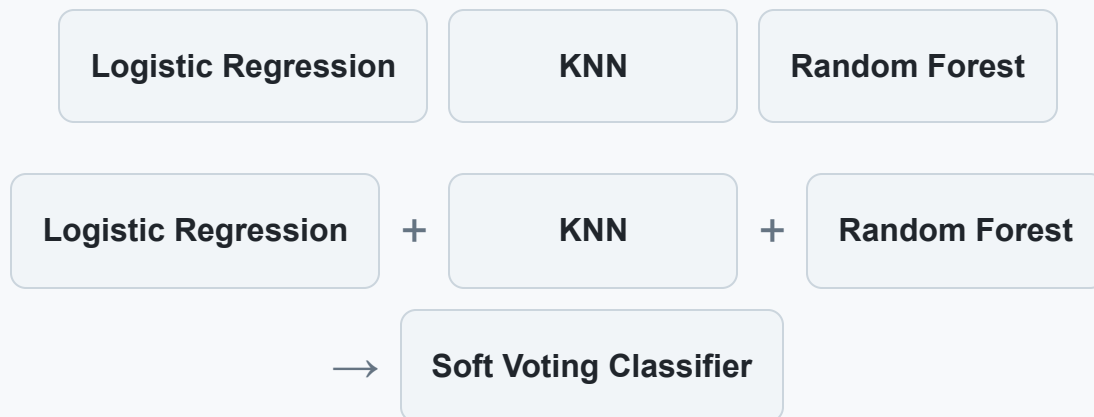
```
accuracy_score(y_test, y_pred_gb)  
  
recall_score(y_test, y_pred_gb)
```

```
classification_report(y_test, y_pred_gb)
```

10. Voting Classifier

A Voting Classifier is implemented as an ensemble model combining Logistic Regression, KNN, and Random Forest.

Ensemble Components



Model Configuration

```
voting_classifier_model = VotingClassifier(  
    estimators=[  
        ("lr", LogisticRegression(  
            max_iter=1000,  
            solver="liblinear"  
        )),  
        ("knn", KNeighborsClassifier(  
            n_neighbors=5  
        )),  
        ("rf", RandomForestClassifier(  
            n_estimators=200,  
            random_state=42  
        ))  
    ],  
    voting="soft"  
)
```

Component	Configuration
Logistic Regression	max_iter=1000, solver="liblinear"
KNN	n_neighbors=5
Random Forest	n_estimators=200, random_state=42
Voting Strategy	Soft voting

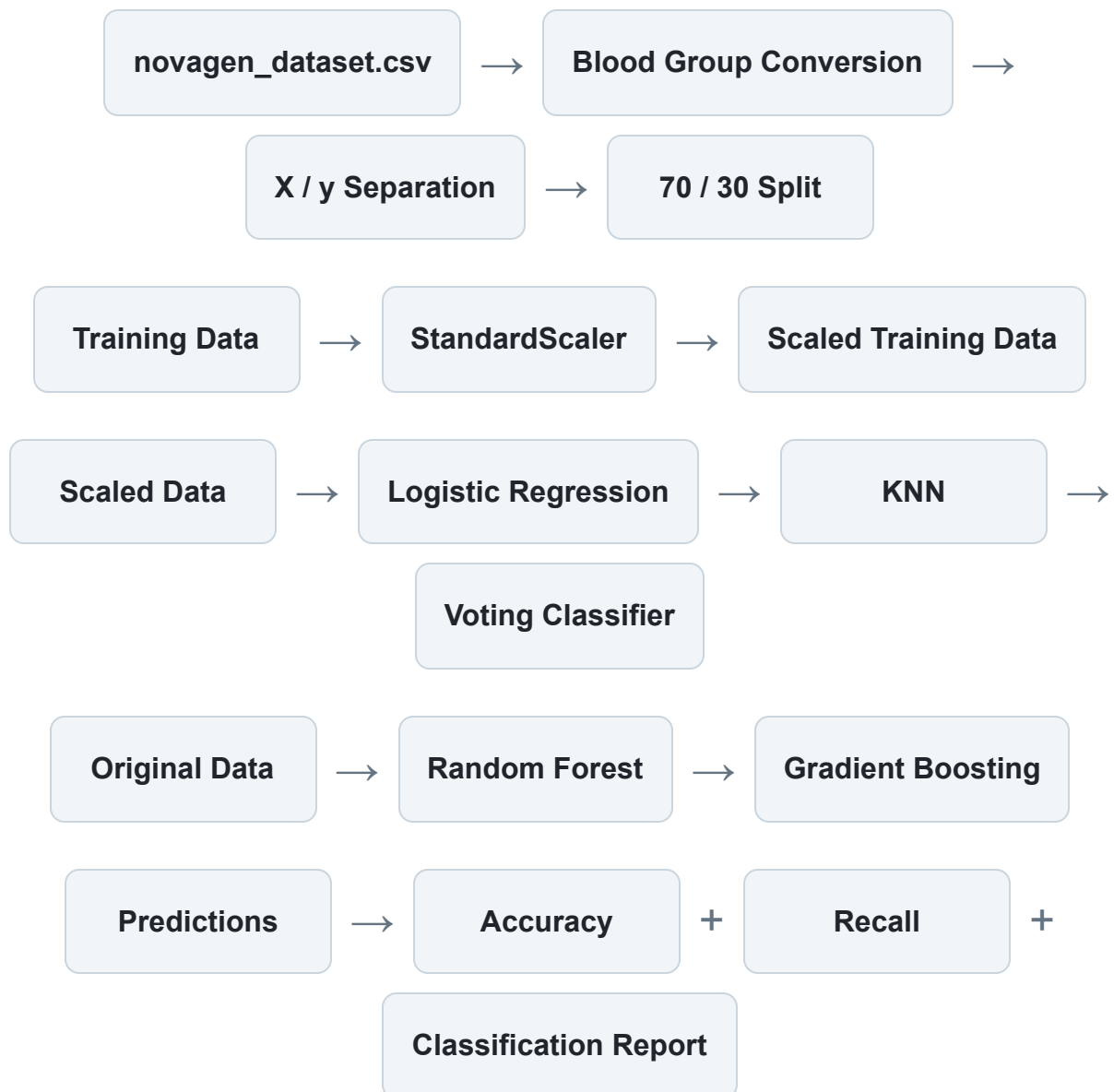
Training and Prediction

```
voting_classifier_model.fit(X_train_scaled, y_train)

y_pred_vote = voting_classifier_model.predict(X_test_scaled)
```

The Voting Classifier is fitted using the scaled training data and predictions are generated from the scaled test data.

11. Complete Machine Learning Workflow



12. Model Comparison

The code evaluates five classification approaches using accuracy, recall, and classification reports.

Model	Accuracy	Recall	Classification Report
Logistic Regression	Not Available	Not Available	Generated
KNN	Not Available	Not Available	Generated
Random Forest	Not Available	Not Available	Generated
Gradient Boosting	Not Available	Not Available	Generated
Voting Classifier	Not Available	Not Available	Generated

The actual printed metric values are not included in the provided code, so numerical results are not available in this documentation.

13. Evaluation Metrics

Accuracy

Accuracy measures the proportion of test predictions that are classified correctly.

```
accuracy_score(y_test, y_pred)
```

Recall

Recall measures how many actual positive samples are correctly identified by the classifier.

```
recall_score(y_test, y_pred)
```

Classification Report

A classification report is generated for every implemented model. The report contains classification performance information for the target classes.

```
classification_report(y_test, y_pred)
```

14. Implementation Characteristics

Component	Implementation in Provided Code
Missing Value Handling	Not Available
Duplicate Removal	Not Available
Outlier Treatment	Not Available
Feature Selection	Not Available
Dimensionality Reduction	Not Available
Hyperparameter Search	Not Available
Cross Validation	Not Available
Feature Importance	Not Available
Confusion Matrix Plot	Not Available
Model Persistence / Saving	Not Available

15. Technical Notes

- StandardScaler is explicitly used for Logistic Regression, KNN, and the Voting Classifier.
- Random Forest and Gradient Boosting are trained using the original train/test feature matrices rather than the scaled matrices.
- The standalone Random Forest model uses `oob_score=True`, but the OOB score is not printed or evaluated in the provided code.
- The standalone Random Forest configuration does not explicitly provide a `random_state`.
- The Random Forest inside the Voting Classifier uses `random_state=42`.
- The Voting Classifier uses `voting="soft"`, combining the probability outputs of its constituent classifiers.
- The Voting Classifier is trained using the scaled feature matrices, including its Random Forest component.
- The imported Seaborn library is not used in any operation shown in the provided code.

16. Technologies and Libraries Used

Technology / Library	Usage in Project
Python	Machine learning implementation
Pandas	Dataset loading and DataFrame operations
Seaborn	Imported but not used in the provided code
Scikit-learn	Data splitting, preprocessing, models, ensemble learning and evaluation
StandardScaler	Feature scaling
LogisticRegression	Linear classification model
KNeighborsClassifier	Nearest-neighbor classification
RandomForestClassifier	Random Forest ensemble classification
GradientBoostingClassifier	Gradient Boosting classification
VotingClassifier	Soft voting ensemble